

TemaFinale26 – Sprint 1 (v1)

Introduction

Questo documento (template del corso) parte dall'**analisi del problema** del **cargoservice** — *di che tipo* sono i componenti, *come* interagiscono e *quale* comportamento ci si attende — e arriva al **sottosistema eseguibile** dello Sprint 1 (sezioni *Project/Testing*). Requisiti e analisi dei requisiti sono in [Sprint 0 \(v1\)](#); qui si analizza la **natura** dei componenti e le **interazioni**, e si **realizza** l'incremento dello Sprint 1.

Goal dello Sprint 1 (sottoinsieme del testo del committente): oltre al core (*"the cargoservice uses the cargorobot to move the container from the IOPort to the slot5 ... and then to the reserved slot"*), realizzare l'**aggancio del sensor dell'IOPort** e il **timeout**: *"the customer must move the container in the sensor area within a prefixed amount of time (e.g. 30 secs), otherwise the system becomes disengaged"*.

Requirements

Il testo esatto del committente e' riportato in [Sprint 0 \(v1\) – Requirements](#). In sintesi, per il core: ricevuta una richiesta valida (pushbutton), il cargoservice riserva uno slot e usa il cargorobot per portare il container IOPort→slot5(marcatura)→slot riservato, poi torna a HOME.

Requirement analysis

Le entita' del dominio (Container, Slot, Hold, IOPort, sensor, marker, LED, cargorobot), la formalizzazione dell'area e le user stories sono in [Sprint 0 \(v1\) – Requirement analysis](#). Qui se ne analizza la **natura come componenti software** e le **interazioni**.

Problem analysis

Perche' qak (QActor): motivazione

La scelta del modello a **attori** non e' arbitraria: il cargoservice e' insieme **proattivo** (guida una sessione di carico in piu' passi mantenendo uno stato) e **reattivo** (pushbutton, sensor, marker, timeout). Questa combinazione «macchina a stati + reazione a messaggi/eventi + temporizzazione» e' cio' che QActor rende di prima classe:

Esigenza del problema	Costrutto QActor
comportamento a fasi con memoria	State/Transition + variabili
richiesta con risposta (pushbutton, robot)	whenRequest/replyTo ; request/whenReply
notifiche asincrone (sensor, marker)	whenEvent

guasto che interrompe da qualunque stato	whenInterruptEvent
timeout dei 30 s	whenTime

Di che tipo sono i componenti? (pojo / service / attore)

- **POJO**: oggetto passivo, invocato in modo sincrono; incapsula dati/stato.
- **Service**: espone operazioni e *risponde* a richieste; reattivo, non ha piano proprio.
- **Attore (QActor)**: autonomo, con stato, proattivo + reattivo, comunica solo per messaggi.

Entita'	Discussione	Natura
cargoservice	guida una sessione in piu' passi e reagisce a eventi: un pojo/service non basta	attore
cargorobot	fornito; raggiunge una posizione su comando (vedi sotto)	service esterno
sensor dell'IOPort	fornisce la distanza D; il sonar e' una sua parte ; natura da chiarire	sorgente di notifiche — <i>aperto</i>
IOPort	pushbutton (sorgente di richiesta = requisito) + display (output; forse web-gui)	pushbutton + display
marker	segnala il completamento	service/attore (evento)
LED	riflette lo stato engaged	indicatore di stato
Hold	stato del dominio, invocato in modo sincrono	POJO

Quale software per il cargorobot, e perche'

Il committente fornisce **piu' componenti** per il DDR: RobotObj26 (POJO, mosse elementari: troppo basso livello), RobotService26 (service, ancora a livello di mosse), robotsmart (service con moverobot(X,Y) orientato all'obiettivo + A*). Il problema chiede di **raggiungere una posizione** (IOPort, slot5, slot, HOME): e' l'astrazione di moverobot(X,Y). Si sceglie quindi, **motivandolo**, robotsmart come service esterno di cui il cargoservice e' client (come boundaryworker usa robotservice). La versione esatta e' rifinitura di Progetto.

Modello delle interazioni

Ruoli, flussi e *natura* di ciascuno (non una semplice lista di messaggi):

```

    richiesta di carico / risposta
customer <-----> CARGOSERVICE <----> cargorobot (raggiungi posizione / esito)
    (pushbutton + display)           | ^
    presenza container / guasto sensor --+ +-- marker (marcatura completata)
                                   |
                                   stato hold (display/web-gui) ; engaged (LED)

```

Flusso	Natura	Altitudine
pushbutton → cargoservice	request/reply	requisito
sensor → cargoservice	notifica → evento (alt.: dispatch)	analisi (motivata)
marker → cargoservice	notifica "fatto" → evento	analisi
cargoservice → cargorobot	request/reply (moverobot)	contratto robotsmart
cargoservice → display/LED	output di stato	progetto / aperto

Il **trasporto** (TCP/CoAP/MQTT) non e' deciso qui: e' Progetto.

Formalizzazione (qak) delle interazioni di base

L'interazione request/reply del pushbutton (un **requisito**) e il contratto del cargorobot si formalizzano come **interfacce di messaggi** (non realizzazione):

```
Request loadrequest : loadrequest()
Reply answer       : answer(ESP) for loadrequest   ESP = reserved(SLOT)||retrylater|reject

Event containerInPlace : containerInPlace(WHERE)    // notifica logica del sensor dell'IOPort

Request moverobot    : moverobot(TARGETX, TARGETY, STEPTIME)    // contratto di robotsmart
Reply moverobotdone  : moverobotok(ARG)                        for moverobot
Reply moverobotfailed: moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

Il **sensor dell'IOPort** e' una **sorgente di notifiche**: la presenza del container (sonar: $D < D_{FREE}/2$ per $\sim 3s$) e' astratta nell'evento **logico** containerInPlace (il formato/trasporto fisico e' Progetto; distinto dal sonaralarm di WEnv, estraneo all'app). La FSM completa (stati, withobj, codice) e' **realizzazione**: vedi *Project* e il modello [cargoservice26.qak](#).

Comportamento atteso (livello di problema)

disponibile -> (richiesta valida) ENGAGED: riserva slot, LED on
-> attende il container nell'area del sensor (max 30s):
 . containerInPlace -> cargorobot: IOPort -> slot5 (marcatura) -> slot riservato
 -> ritorno a HOME -> disponibile
 . scadono 30s -> DISENGAGE: libera lo slot, LED off -> disponibile
Vincoli: una sola richiesta per volta; stiva piena -> reject; IOPort occupato/Out of service -> retrylater.

Lo **Sprint 1** aggiunge al core proprio i due rami in grassetto: l'attesa del containerInPlace dal sensor e il **timeout 30s** → **disengage**.

Core business e strategia incrementale

Iterazione	Cosa realizza
Sprint 0 (core)	flusso nominale (container assunto presente); display/LED come log
Sprint 1 (questo)	realizzato : sensor dell'IOPort (containerInPlace) + timeout 30s → disengage
Sprint 2	Out of service + display web-gui

Decisioni e domande aperte

Tema	Altitudine	Stato
deposito = il robot raggiunge la posizione (no attuatore di presa)	analisi	confermato

cargorobot = robotsmart	analisi (motivata)	versione → Progetto
sensor dell'IOPort	analisi → progetto	interfaccia logica definita (containerInPlace); formato fisico del sonar → Progetto
natura del display (web-gui?)	analisi	aperta (Sprint 2)
trasporto (MQTT/TCP/CoAP)	progetto	rinviato

Test plans

Il cargoservice mantiene lo stato della stiva: il test e' **automatizzabile** (PASS/FAIL). Il caso significativo (**TP1, carico nominale**) verifica a fine ciclo: slot riservato occupato, robot a HOME, cargoservice disponibile. Casi aggiuntivi: stiva piena (reject), IOPort occupato/Out of service (retrylater), timeout (disengage). Dettaglio in [Sprint 0 \(v1\) – Test plans](#).

Project

Il modello logico e' realizzato nel **sottosistema eseguibile** dello Sprint 1 ([TemaFinale26/Sprint1/cargoservice26](#)): la **FSM qak** [cargoservice26.qak](#) con i POJO [Hold](#), [DisplaySim](#), [LedSim](#). Il cargoservice e' un QActor client di robotsmart (ExternalQActor); struttura e build modellate su robotsmart26usage.

Incremento dello Sprint 1 rispetto al core, con la fonte di ogni costruito qak:

Elemento	Costrutto qak	Fonte del pattern
notifica del sensor dell'IOPort	Event containerInPlace + whenEvent	firefly100.qak, griddisplay.qak
attesa engaged con scadenza	State waitContainer: whenEvent containerInPlace -> goToIOPort & whenTime 30000 -> disengage	firefly100.qak (whenTime+whenEvent)
timeout → libera lo slot	State disengage → hold.freeSlot(Slot), LED off	requisito (e.g. 30 secs)
sensor simulato	QActor iosensor: dopo un delay emit containerInPlace	demoSendReceiveEmit.qak
pushbutton simulato	QActor pushbutton: request cargoservice -m loadrequest	request/reply standard

I dispositivi (sensor, pushbutton) sono **simulati** per **eseguire** il modello senza hardware; il cargoservice dipende solo dall'informazione scambiata, quindi sono sostituibili dai dispositivi reali (es. il PicoW) senza modificare la FSM.

Testing

Il POJO Hold e' **compilato ed eseguito** con il test [TestHold](#) — **20 PASS, 0 FAIL**: logica reserved/retrylater/reject (TP2/TP3), slot5 mai riservato, posizioni IOPort/HOME e, nuovo nello Sprint 1, il **disengage da timeout** (TP4: lo slot riservato torna libero e riservabile).

```
> javac -d build/testclasses utils/domain/Hold.java utils/test/TestHold.java
> java -cp build/testclasses test.TestHold
PASS T8 disengage: slot liberato
PASS T8 disengage: lo slot e' di nuovo riservabile    (... 20 asserzioni ...)
TestHold (Sprint1): 20 PASS, 0 FAIL
```

La generazione del modello qak (plugin QAK), la build e l'esecuzione end-to-end di TP1 con robotsmart sono il passo successivo ([cargoservice26/README](#)).

Deployment

Vedi [Sprint 0 \(v1\) – Deployment](#): Docker (VR+GUI+mosquitto) + robotsmart26 (8020) + cargoservice26 (8030). Con mosquitto in Docker usare l'IP reale del PC.

Maintenance

Poiche' il cargoservice dipende solo dall'*informazione* scambiata (eventi logici via iosensor), sostituire i dispositivi simulati con l'hardware reale non modifica il cargoservice.



By Alexander Kreuzer

Email: Alexander.kreuzer@studio.unibo.it

GIT repo: https://github.com/Alexing-uni/ISS26_Alexander_Kreuzer